

Beginner's Guide to MCP



A Must Read



The Challenge with Context in LLMs

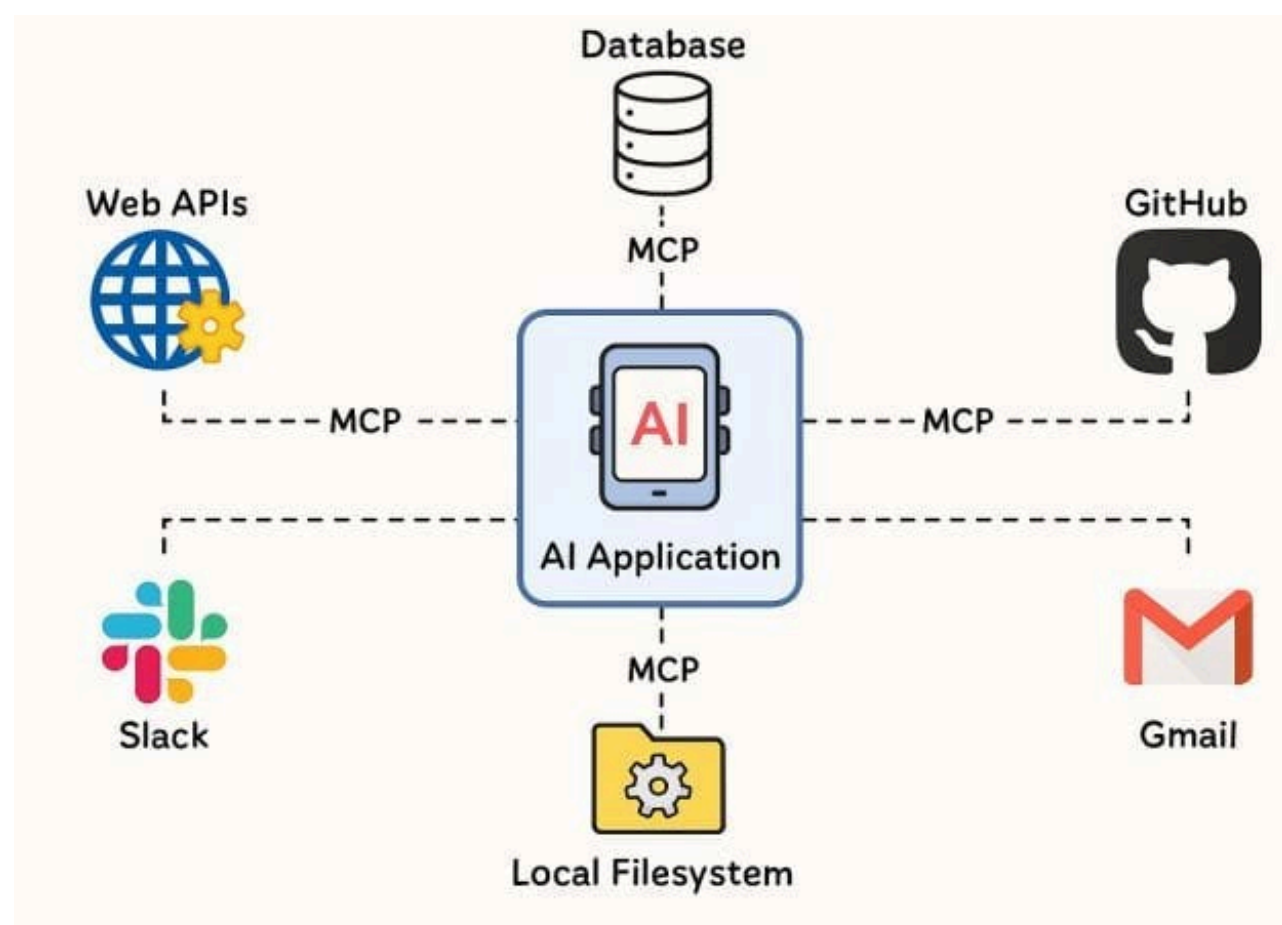
LLMs like Claude 4 can process massive context windows. This allows them to understand long documents or codebases in a single pass.

However, while having such a large context window is impressive, the real challenge lies in effectively managing and providing relevant context to these models in real-time.



The Solution: MCP

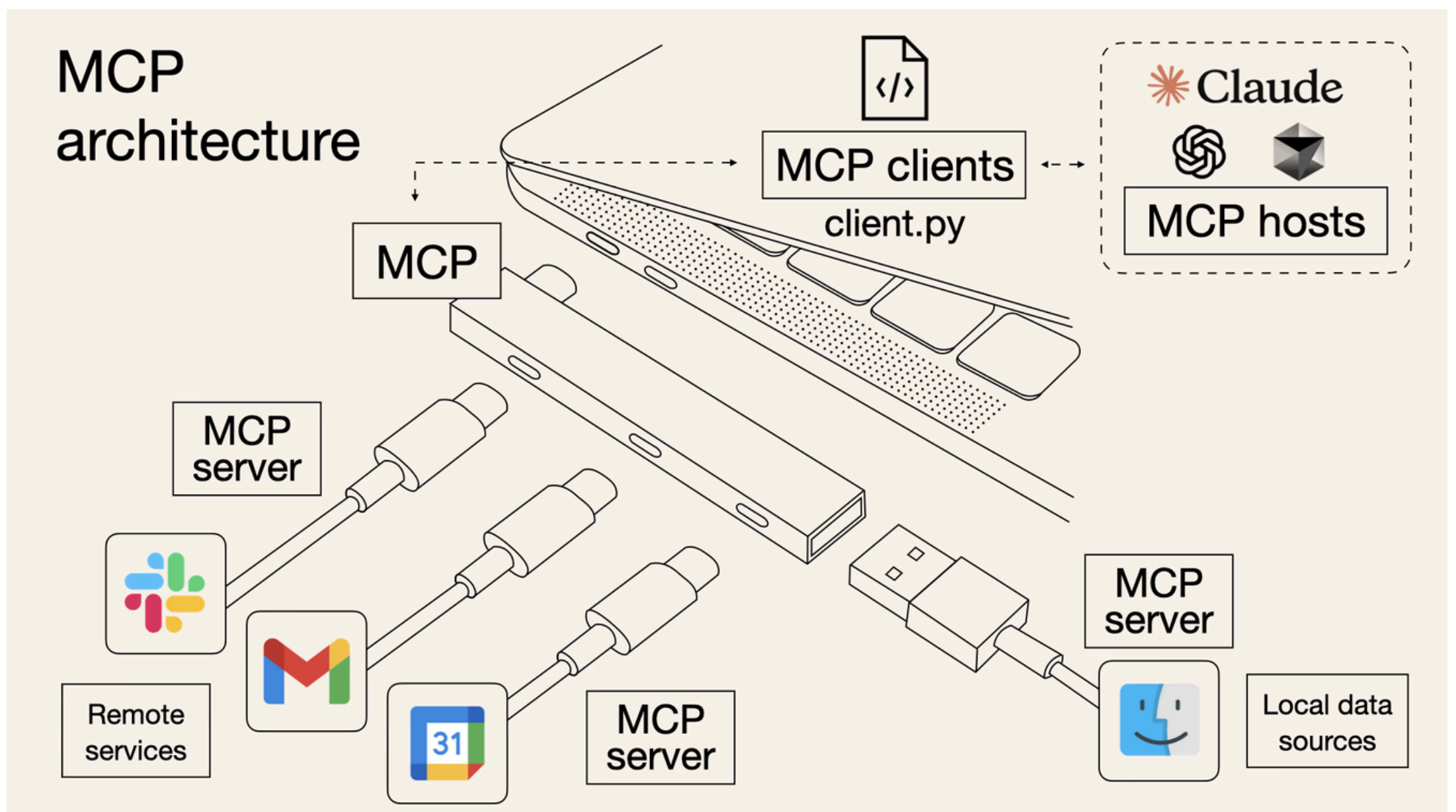
MCP simplifies how LLMs connect to external data, systems, and tools. Think of MCP as the USB-C for AI, a universal connector that allows LLMs to seamlessly access and integrate context without cumbersome and redundant custom code.



How MCP Works?

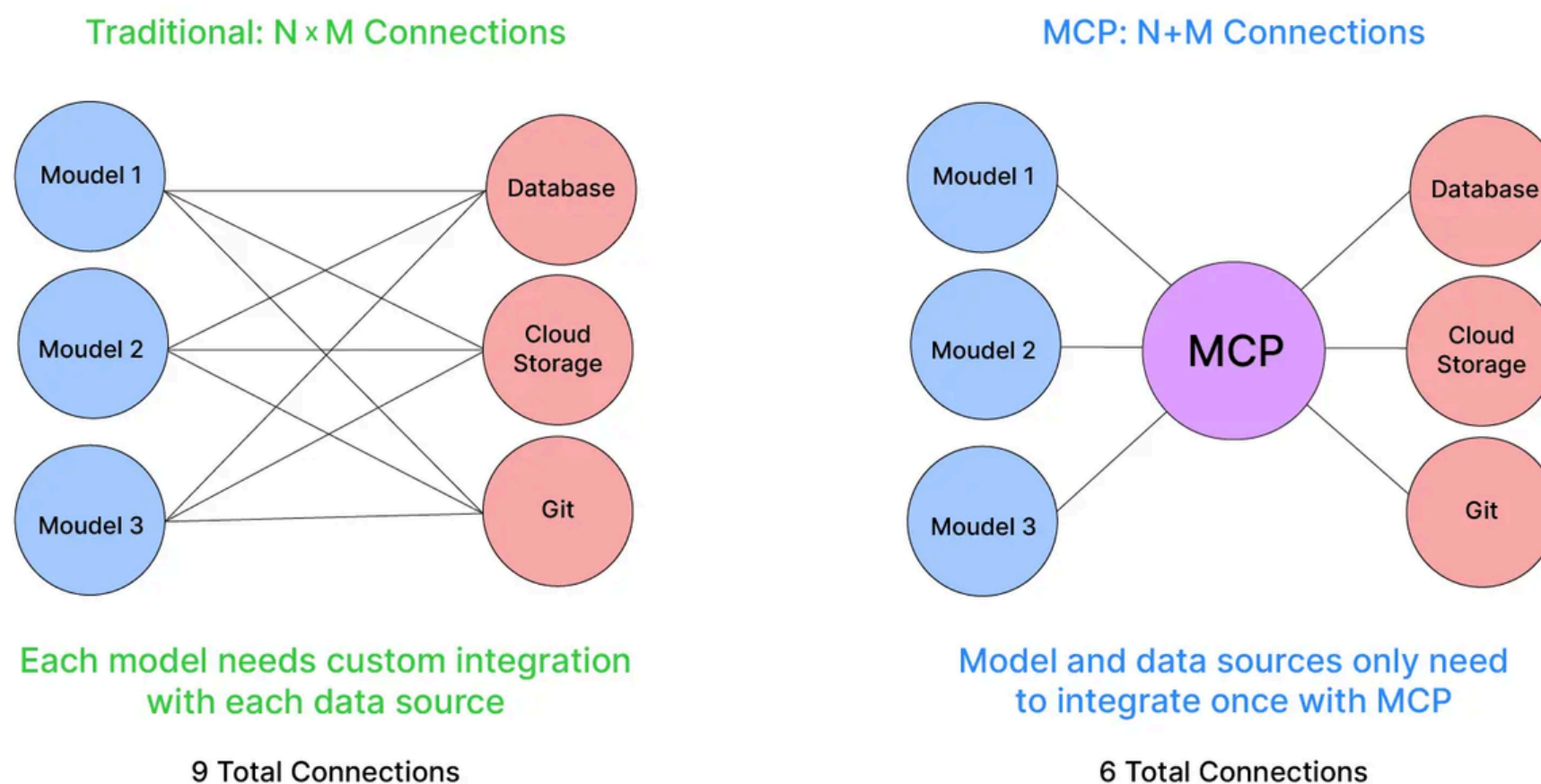
MCP divides external context into three main components:

- **Resources:** These are read-only data sources (e.g., documents, API responses) that provide context to the LLM when needed.
- **Tools:** These are functions or actions (e.g., sending an email, running a computation) that the model can invoke during its operation.
- **Prompts:** Predefined instructions or templates that guide the LLM's behavior, providing consistent context for tasks like Q&A or summarization.



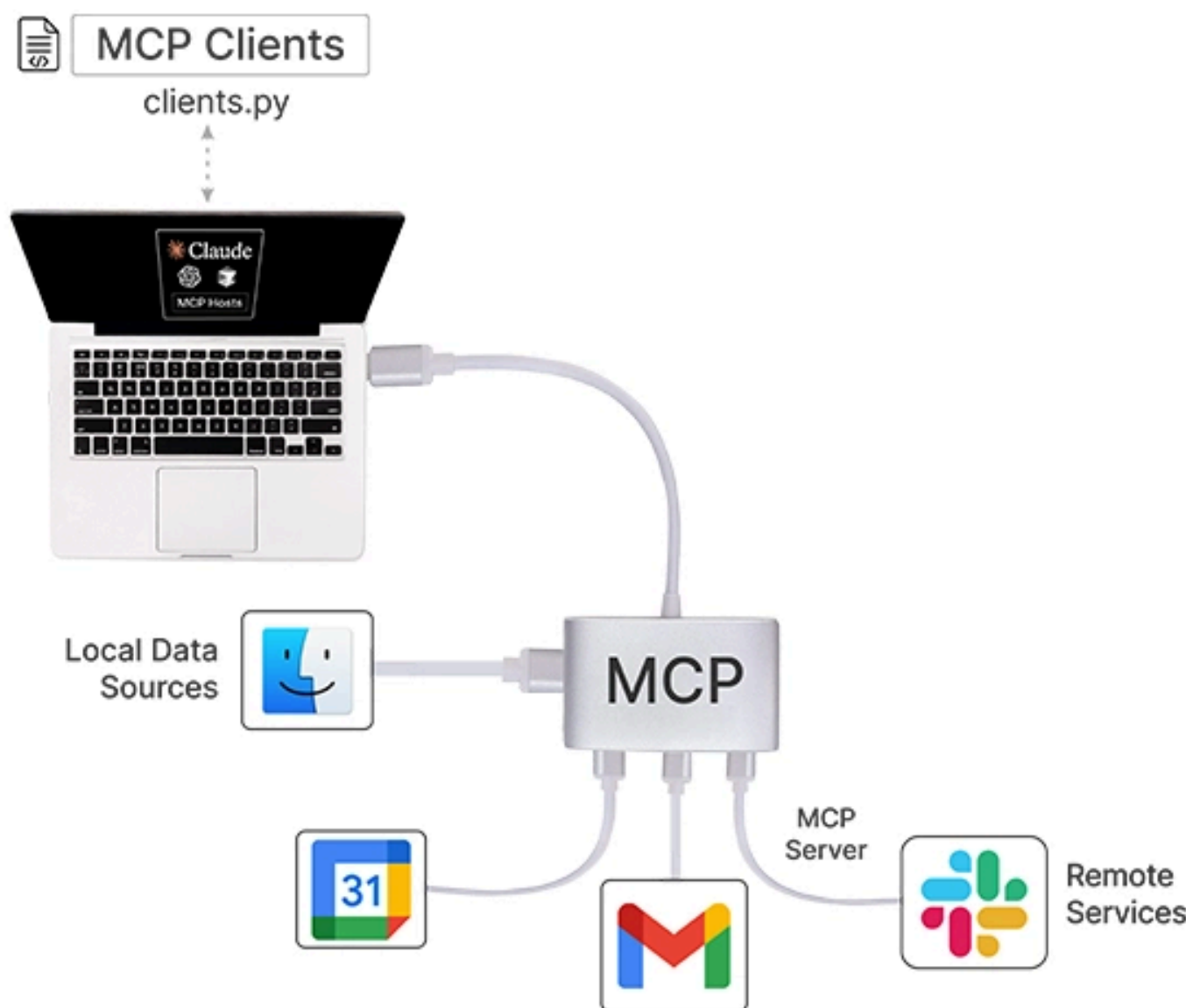
MCP vs. Traditional Approaches

Before MCP, LLMs were integrated with external systems through custom, often inefficient methods:



- **Manual Prompt Engineering:** Lacking standardization, this required creating large, complex prompts for each new data source.
- **Retrieval-Augmented Generation (RAG):** A system might use a vector search to pull in relevant context and append it to a prompt, but it lacked flexibility and was resource-intensive.

MCP Architecture: How Data Flows?



Host: The LLM application (e.g., Claude's desktop app).

MCP Client: A library within the host that connects to external data sources.

MCP Server: The external system (database, API, or tool) providing resources, tools, and prompts in MCP format.

- **Initialization:** The client connects to the server, verifying compatibility.
- **Capability Discovery:** The client checks what tools, resources, and prompts the server can offer.
- **Context Provision:** The model requests and retrieves necessary context or tools based on the user's query.
- **Action and Integration:** The model executes tasks (e.g., querying an API) and integrates the results into its response.